Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Advance Training Algorithms: Variance Reduction

Variance Reduction



- Recall m_t is a new estimator of $\nabla f(w_t)$ with some bias and lower variance than g_t.
- How can we get better variance reduction? One idea is Control variates from Statistics
- Let G be an unbiased estimator of F. Consider X and Y s.t. $\mathbb{E}[Y] = \mathbb{E}[X]$.
- Consider the new estimator

$$Z = G + \beta(X - Y) \tag{1}$$

- Let us calculate the mean and variance of Z

$$\begin{aligned} \mathbb{E}[Z] &= \mathbb{E}[G + \beta(X - Y)] = \mathbb{E}[G] + \beta\mathbb{E}[x] - \beta\mathbb{E}[Y] = F \\ \mathbb{E}\|Z - F\|^2 &= \mathbb{E}\|G - F\|^2 + \beta^2\mathbb{E}\|X - Y\|^2 + 2\beta\mathbb{E}[\langle G - F, X - Y\rangle] \end{aligned} \tag{2}$$

- That is, if $X - Y$ is negatively correlated with G such that

$$2\mathbb{E}[\langle G - F, X - Y\rangle] \leq -\beta\mathbb{E}\|X - Y\|^2, \tag{3}$$

we may get a reduction in variance and retain unbiasedness.

Control variates in Optimization



- Consider the following update vector and iterates $w_{t+1} = w_t - \eta u_t$

$$Z = G + \beta(X - Y), \qquad u_t = \nabla \ell(w_t, z_t) + 1 \cdot (\nabla f(v) - \nabla \ell(v, z_t)) \tag{4}$$

  for some reference point v
- Since $\mathbb{E}[\nabla \ell(v, z_t)] = \nabla f(v)$, and $\mathbb{E}[\ell(w_t, z_t)] = \nabla f(w_t)$, u_t is unbiased.
- Let us assume both f and ℓ are L-smooth. Using basic techniques we can show

$$\mathbb{E}\|u_t - \nabla f(w_t)\|^2 \lesssim L\|w_t - v\|^2 \tag{5}$$

- Obviously, $v = w_t$ leads to the least variance, but too costly.
- Main idea: Only update the reference once in a while to trade some variance for computational efficiency

SVRG Algorithm



Consider the ERM objective $f(w) = \frac{1}{N}\sum_{i=1}^{N} \ell(w, z_i)$

- Initialize w_1 and $v_1 = w_1$, set $e = 1$ (the epoch counter)
- Compute $\nabla f(v_1)$ $\mathcal{O}(N)$ gradient computation
- for $t = 1, \dots T$
  - sample a batch $z_t \sim p_z$
  - form the update vector $u_t = \nabla \ell(w_t, z_t) + \nabla f(v_e) - \nabla \ell(v_e, z_t)$ 2 gradient computation
  - parameter update $w_{t+1} = w_t - \eta u_t$
  - if $t \equiv 0 \mod N$ (finishing one epoch, i.e., going over all training data)
    - $e \leftarrow e + 1$ increase epoch counter
    - $v_e = w_t$ update the reference
    - update the reference gradient $\nabla f(v_e)$ $\mathcal{O}(N)$ gradient computation every N iterations

Thus, if $T = EN$, the total gradient computation is $3EN = 3T$ (i.e., 3 times SGD's)

SVRG vs. SGD vs GD



- Recall, the oracle complexity of GD to find a stationary solution was $\mathcal{O}(N/\epsilon)$
- Recall, the oracle complexity of SGD to find a stationary solution was $\mathcal{O}(1/\epsilon^2)$
- It turns out the oracle complexity of SVRG to find a stationary solution is $\mathcal{O}(N + \frac{N^{2/3}}{\epsilon})$
- SVRG is better than GD if

$$N + \frac{N^{2/3}}{\epsilon} \leq N/\epsilon \quad \equiv \quad \epsilon^{-2} > \frac{N^{1/3}}{N^{1/3} - 1} \approx 1 \quad \text{always the case} \tag{6}$$

- SGD is better than GD if

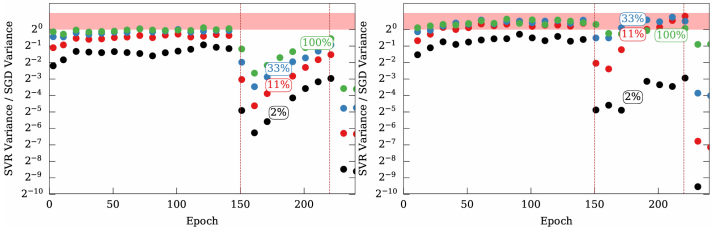
$$1/\epsilon^2 \leq N/\epsilon \quad \equiv \quad \epsilon^{-1} < N \quad \text{almost always the case in DL} \tag{7}$$

- SVRG is better than SGD if

$$N + \frac{N^{2/3}}{\epsilon} \leq 1/\epsilon^2 \quad \equiv \quad \epsilon^{-1.5} > N \quad \text{almost never the case in DL!} \tag{8}$$

Other shortcomings of SVRG-based methods

- For simple ML problems, not an issue, but for DL when using batch normalization, dropout, and data augmentation, this leads to ineffectiveness of variance reduction.
- The issue can be attributed to the finite-sum assumption $f(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(w, z)$.
- SVRG assumes whenever a z is sampled, the loss $\ell(w, z)$ is returned. Not true in DL!
- let ρ denote the randomness due to normalization, dropout, and data augmentation. In fact, $\ell(w, z) = \mathbb{E}_\rho[\tilde{\ell}(w, z_i, \rho)]$. Consequently, the sum is infinite.



(c) Small ResNet (d) ResNet-110

Figure 2: The SVRG to SGD gradient variance ratio during a run of SVRG. The shaded region indicates a variance *increase*, where the SVRG variance is worse than the SGD baseline. Dotted lines indicate when the step size was reduced.

Role of Data Augmentation



- In practice, to improve generalization and robustness, we not only use a sample z (say an image), but also its transformations (e.g., cropping, rotation, flipping), in training.
- In many DL packages, a transformation is chosen randomly any time we process a given z.
- Consider image $\tilde{z}$ and assume we will use it in iteration t
- Recall $u_t = \nabla \ell(w_t, \tilde{z}) + \nabla f(v_e) - \nabla \ell(v_e, \tilde{z})$. Also recall $\nabla f(v_e)$ is calculated outside an epoch while $\nabla \ell(v_e, \tilde{z})$ is calculated inside the epoch.
- $\tilde{z}$ appears with transformation T_1 in orange terms and T_2 in the blue term, Implying we may incur an estimation error due to bias and increased variance.

Role of Data Augmentation (Cont'd)

- A solution: Transformation Locking, meaning store the transformation used outside epoch and reused it inside epoch
- Nontheless, as model updates, variance goes up, meaning, v_e not a good reference anymore.

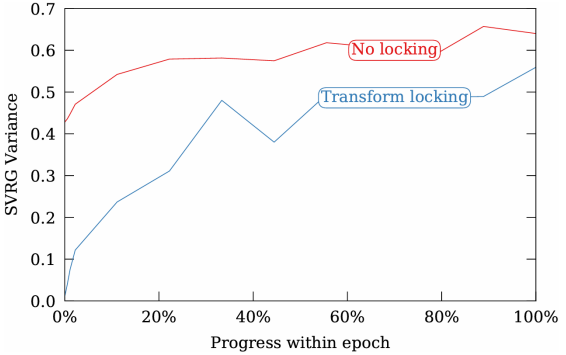


Figure 1: Variance within epoch two during LeNet training on CIFAR10.

Role of Dropout and Batch Normalization



- Dropout issue is similar to data augmentation as outride and inside epoch, different sparsity patterns are used. So, we can implement a similar locking solution.
- Issue with BN is complicated and two folds
- First, $\nabla \ell(w_t, \tilde{z})$ will now depends on other samples too, not just $\tilde{z}$

$$\nabla \ell(w_t, \tilde{z}; B_1) \neq \nabla \ell(w_t, \tilde{z}; B_2), \qquad B_1 \neq B_2 \tag{9}$$

  Implication: increased bias and variance in SVRG estimator due to the epoch-based structure
- Secondly, recall during inference, we use the statistics of the entire data to calculate the mean and variance of each layer's activation from mini-batch statistics found during training:

$$\mu_{\mathcal{D}} \leftarrow \beta \mu_{B_t} + (1-\beta)\mu_{\mathcal{D}}, \qquad \sigma^2_{\mathcal{D}} \leftarrow \beta \sigma^2_{B_t} + (1-\beta)\sigma^2_{\mathcal{D}} \tag{10}$$

- Default implementation won't compatible with SVRG as we compute the $(\mu_{B_t}, \sigma^2_{B_t})$ of a specific batch at both w_t and v_e.
- Solution: make sure statistics at v_e not used (batch reset)

Fixing SVRG



- In summary, we have issues due to the epoch-based structure and the finite-sum assumption and their incompatibility with DL operations.
- Ideally, we want to update the reference on the fly in each iteration. But doing so may be costly (e.g., requiring full gradient)
- Solution: Approximate control variates! Recall we said consider X and Y s.t. $\mathbb{E}[Y] = \mathbb{E}[X]$ and the estimator

$$Z = G + \beta(X - Y) \tag{11}$$

- Let us relax the condition such that now $\mathbb{E}[Y] \approx \mathbb{E}[X]$. Implication: some bias which we may control by finding a good β. Seems like Bias-variance tradeoff then! Connection to momentum?

SGDm as Approximate control variates



- Let us recall our SGDm update vector: $m_t = \beta m_{t-1} + (1-\beta)\ell(w_t, z_t)$, $m_0 = 0$
- Let us do a rearrangement

$$m_t = \nabla\ell(w_t, z_t) + \beta(m_{t-1} - \ell(w_t, z_t)), \qquad Z = G + \beta(X - Y) \tag{12}$$

- Intuitively, m_{t-1} is a good approximation of $\nabla f(w_{t-1})$ while $\nabla\ell(w_t, z_t)$ is a good approximation of $\nabla f(w_t)$
- under slow evolution assumption $w_{t-1} \approx w_t$ so

$$m_{t-1} \approx \nabla f(w_{t-1}) \approx \nabla f(w_t) \approx \ell(w_t, z_t) \tag{13}$$

  Hence SGDm as like Approximate control variates
- This discussion helps us to develop a better method by using something better to reduce the approximation bias

De-Biasing SGDm



- Previously, we talked about a de-biasing step: $m_t = \beta m_{t-1} + (1-\beta)\ell(w_t, z_t)$, $\hat{m}_t = m_t/(1-\beta^t)$
- Consider our previous discussion: m_{t-1} is a good approximation of $\nabla f(w_{t-1})$ while $\nabla \ell(w_t, z_t)$ is a good approximation of $\nabla f(w_t)$
- What if we change $\ell(w_t, z_t)$ to something that is a good approximation of $\nabla f(w_{t-1})$ instead?
- A candidate: $\nabla \ell(w_{t-1}, z_t)$ leading to the method

$$w_{t+1} = w_t - \eta m_t, \qquad m_t = \nabla \ell(w_t, z_t) + \beta(m_{t-1} - \nabla \ell(w_{t-1}, z_t)) \tag{14}$$

De-Biasing SGDm vs SVRG



- Our new estimator $m_t = \nabla \ell(w_t, z_t) + \beta(m_{t-1} - \nabla \ell(w_{t-1}, z_t))$
- Compared with SVRG's $u_t = \nabla \ell(w_t, z_t) + 1 \cdot (\nabla f(v) - \nabla \ell(v, z_t))$
  - Our reference point is now w_{t-1} which is much closer to w_t than v
  - We approximately calculate the full gradient at reference by using m_{t-1},
  - all calculations inside epoch as opposed to SVRG
  - using two gradient calculations in each iteration, like SVRG
  - Using $\beta < 1$ to tame the approximation bias as opposed to $\beta = 1$ in SVRG

Convergence Analysis



Theorem

Assume $f(w) = \mathbb{E}[\ell(w, z)]$ and $\ell(w, z)$ are both L-smooth. Furthermore, assume we have access to an SFO and that $z_1, \ldots, z_T$ are statistically independent. Then, the de-biased SGDm with learning rate $\eta = \mathcal{O}(\frac{1}{T^{1/3}})$ and momentum parameter $1 - \beta = \mathcal{O}(\frac{1}{T^{2/3}})$ satisfies $\mathbb{E}\|\nabla f(\hat{w})\|^2 \simeq \mathcal{O}(\frac{1}{T^{2/3}})$.

- Recall for SGD: $\eta = \mathcal{O}(\frac{1}{T^{1/2}})$, $\mathbb{E}\|\nabla f(\hat{w})\|^2 \simeq \mathcal{O}(\frac{1}{T^{1/2}})$
- Recall for SGDm: $\eta = \mathcal{O}(\frac{1}{T^{1/2}})$, $1 - \beta = \mathcal{O}(\frac{1}{T^{1/2}})$, $\mathbb{E}\|\nabla f(\hat{w})\|^2 \simeq \mathcal{O}(\frac{1}{T^{1/2}})$
- Thus, we get a provably better rate compared to both SGD and SGDm
- We are then using larger learning rates and momentum parameters, thanks to the reduced approximation bias.
- Recently shown optimal under the stated assumptions

Proof Sketch



- Like SGDm proof, we consider and analyze the evolution of the approximation error $e_t = m_t - \nabla f(w_t)$ and optation

$$\mathbb{E}\|e_t\|^2 \leq \beta \mathbb{E}\|e_{t-1}\|^2 + \text{ other terms} \tag{15}$$

- We then define a potential function (again like SGDm proof)

$$\Phi_t = \mathbb{E}[f(w_t)] - f^* + C_e \mathbb{E}\|e_t\|^2 \tag{16}$$

- Study the evolution of potential by summing over its consecutive differences
- When $\beta < 1$ we can find a good constant C_e to ensure convergence.